

PIPELINEIQ // SELF-HEALING MLOPS PLATFORM

Automated End-to-End Machine Learning Lifecycle Engine with SMOTE, MLflow, and Evidently AI Telemetry

B.Tech Thesis Project Codebase & Architectural Documentation

Submitted by Group No. 10

* Maimoona Manzoor (CSE-22-LE-70)

* Syed Maryam Andrabi (CSE-22-LE-74)

* Momin Zahoor (CSE-22-LE-71)

Under the Supervision of: Dr. Sahil Sholla (Assistant Professor, Department of CSE, IUST Kashmir)

Department of Computer Science and Engineering

Islamic University of Science and Technology, Awantipora, Kashmir - 192122 — Spring 2026

□ Table of Contents

1. [Executive Summary & Abstract](#executive-summary--abstract)
2. [Key Features](#key-features)
3. [Four-Layer Architectural Blueprint](#four-layer-architectural-blueprint)
4. [Mathematical Definitions & System Algorithms](#mathematical-definitions--system-algorithms)
5. [Prerequisites & System Requirements](#prerequisites--system-requirements)
6. [Quick-Start Execution Options](#quick-start-execution-options)
 - [Option A: Instant Demo Simulation Mode](#option-a-instant-demo-simulation-mode-zero-backend-setup)
 - [Option B: Production Docker Compose Execution](#option-b-production-docker-compose-execution)
 - [Option C: Native Development Environment Setup](#option-c-native-development-environment-setup)
7. [Step-by-Step User Manual](#step-by-step-user-manual)
8. [REST API Specification](#rest-api-specification)
9. [Database Schema Reference](#database-schema-reference)
10. [Troubleshooting & FAQs](#troubleshooting--faqs)
11. [PDF User Manual Download](#pdf-user-manual)

□ Executive Summary & Abstract

Per academic findings by Sculley et al. (NeurIPS 2015), up to 90% of deployed ML technical debt lies in operational infrastructure—data validation, feature scaling, model drift monitoring, and manual retraining pipelines—rather than model mathematics.

PipelineIQ is an enterprise-grade, cloud-native, self-healing MLOps platform engineered to eliminate the manual operational overhead of maintaining production machine learning pipelines. It integrates automated dataset balancing (SMOTE), multi-model concurrent execution (Random Forest, XGBoost, LightGBM), experiment tracking (MLflow), statistical drift detection (Evidently AI, KS-Test, Population Stability Index), and an autonomous self-healing trigger into a unified reactive dashboard.

✂ Key Features

- **Dynamic Ingestion & SMOTE Balancing:** Automatically detects binary target columns, purges zero-variance/identifier attributes, imputes missing records, applies Quantile Normalization, and balances minority classes using Synthetic Minority Over-sampling Technique (SMOTE).
- **Concurrent Multi-Model Execution:** Concurrently trains Random Forest, XGBoost, and LightGBM models with hyperparameter depth tuning and decision boundary probability calibration (tau sweep from 0.30 to 0.70).
- **MLflow Experiment Tracking:** Formally logs hyperparameter configurations, metrics (Accuracy, F1-Score, ROC-AUC), and champions model registries into local/PostgreSQL MLflow backends.
- **Statistical Covariate Drift Detection:** Calculates 2-sample Kolmogorov-Smirnov (\$p\$-value threshold < 0.05) and Population Stability Index ($\text{PSI} > 0.25$) on incoming inference data to detect covariate shift.
- **Autonomous Self-Healing Retraining:** Automatically responds to drift breaches by spawning background worker threads, generating boosted hyperparameter profiles, retraining candidate models, and performing zero-downtime champion model promotion.
- **Enterprise Security & Authentication:** Built with FastAPI JWT Bearer token authentication, bcrypt password hashing, and optional sync with MLflow User Auth registry.
- **Reactive Real-Time Telemetry UI:** Responsive React + Tailwind CSS dashboard with TanStack Query 5-second polling, optimistic UI updates, and self-healing alert banners.

□ Four-Layer Architectural Blueprint

The platform architecture is divided into four domain-isolated layers:

PipelineIQ Monorepo Structure

```

├── backend/                # FASTAPI + PYTHON MACHINE LEARNING CORE
│   ├── data/
│   │   └── ibm_hr_attrition.csv # Layer 1: Reference tabular dataset
│   ├── ml_engine.py          # Layer 1 & 2: Preprocessing, SMOTE, AutoML, MLflow tracking
│   ├── drift_monitor.py      # Layer 3: Statistical drift monitoring (PSI & KS-Test)
│   ├── main.py               # Layer 3: FastAPI REST API & JWT security layer
│   ├── db.py                 # MongoDB / Motor asynchronous connection layer
│   ├── storage.py            # MinIO / AWS S3 object storage helper
│   ├── requirements.txt      # Backend dependencies
│   └── Dockerfile            # Backend container specification
├── src/                     # REACT + TAILWIND CSS + ZUSTAND FRONTEND
│   ├── components/
│   │   ├── TrainingSetup.jsx # Multi-model concurrent execution modal & dataset drag-and-drop
│   │   ├── DriftMonitor.jsx  # Evidently AI telemetry feed & drift trigger controls
│   │   └── ProtectedRoute.jsx # Auth guard interceptor
│   ├── services/
│   │   └── api.js             # Axios instance with JWT Bearer token interceptor & demo fallback
│   ├── stores/
│   │   └── useAuthStore.js    # Zustand global session & auth persistence
│   ├── views/
│   │   ├── DashboardOverview.jsx # Telemetry dashboard & MLflow run history
│   │   ├── Login.jsx          # Minimalist industrial login screen
│   │   └── Register.jsx       # Operator registration screen
│   ├── App.jsx               # QueryClient provider & route definitions
│   └── Dockerfile            # React Vite web interface container spec
├── docker-compose.yml        # Layer 4: Multi-container production orchestration
├── PipelineIQ_User_Manual.pdf # Compiled printable User Manual PDF
└── README.md                 # Primary project documentation

```

Layer Breakdown

1. Layer 1: Data Ingestion & Preprocessing Layer

Handles multipart CSV dataset uploads, automatic target detection, missing value median/mode imputation, quantile feature transformation, and minority oversampling via SMOTE.

2. Layer 2: AutoML Training & Experiment Tracking Layer

Executes concurrent model builds across Random Forest, XGBoost, and LightGBM, evaluates calibrated F_1 -Score, and logs artifacts to MLflow (<http://localhost:5001>).

3. Layer 3: Serving, Security & Statistical Drift Monitoring Layer

Secures endpoints via JWT Bearer tokens and continuously computes 2-sample Kolmogorov-Smirnov statistics and Population Stability Index (PSI).

4. Layer 4: Containerization & Cloud Orchestration

Orchestrates isolated Uvicorn API workers, React Vite client servers, MongoDB databases, and MLflow tracking servers using docker-compose.yml.

□ Mathematical Definitions & System Algorithms

Algorithm 1: PreprocessAndBalanceSMOTE

$$\text{SMOTE Synthesized Sample: } x_{\text{new}} = x_i + \lambda (x_{\text{zi}} - x_i), \quad \lambda \sim U(0, 1)$$

ALGORITHM 1: PreprocessAndBalanceSMOTE

INPUT: Raw DataFrame D, target_hint

OUTPUT: X_train_resampled, X_test_scaled, y_train_resampled, y_test

BEGIN

Detect target column (target_col) via hint or binary unique count.

Drop zero-variance and unique ID columns.

Impute missing numerical values using Median and categorical values using Mode.

Apply QuantileTransformer(output_distribution='normal') scaling.

If minority class count > 1:

Apply SMOTE(k_neighbors=min(5, count - 1)) to balance training set.

RETURN preprocessed feature and target matrices.

END

Algorithm 2: Multi-Model Concurrent AutoML Training

ALGORITHM 2: TrainConcurrentAutoML

INPUT: dataset_name, models_to_run, target_metric, DataFrame D

OUTPUT: List of model execution metrics & champion model registration

BEGIN

X_train, X_test, y_train, y_test ← PreprocessAndBalanceSMOTE(D)

FOR EACH model IN models_to_run DO

Fit candidate model (Random Forest / XGBoost / LightGBM) on X_train, y_train.

Calibrate decision threshold tau in range [0.30, 0.70] step 0.05.

Compute Accuracy, F1-Score, ROC-AUC.

Log metrics and parameters to MLflow via mlflow.start_run().

END FOR

Select model with maximum target_metric as Champion Model.

Save model run telemetry record into MongoDB database.

END

Algorithm 3: Population Stability Index (PSI) & Kolmogorov-Smirnov Drift

$$\text{PSI} = \sum_{b=1}^B \left| P_{\text{current}}(b) - P_{\text{baseline}}(b) \right| \times \ln \left| \frac{P_{\text{current}}(b)}{P_{\text{baseline}}(b)} \right|$$

ALGORITHM 3: EvaluateDataDrift

INPUT: df_reference, df_current, bins=10

OUTPUT: mean_psi, min_ks_pvalue, is_drifted

BEGIN

FOR EACH numerical column DO

Bin reference and current distributions into 10 quantile intervals.

Calculate column PSI using smoothed percentages.

Execute SciPy 2-sample Kolmogorov-Smirnov test (ks_2samp) -> ks_pvalue.

END FOR

IF mean_psi > 0.25 OR min_ks_pvalue < 0.05 THEN

is_drifted ← TRUE (Covariate Shift Detected)

ELSE

is_drifted ← FALSE (Stable Baseline)

END IF

RETURN mean_psi, min_ks_pvalue, is_drifted

END

Algorithm 4: Autonomous Self-Healing Pipeline Trigger

ALGORITHM 4: TriggerSelfHealingPipeline

INPUT: drift_telemetry, active_models

BEGIN

IF drift_telemetry.is_drifted == TRUE THEN

Set state flag = "AUTONOMOUS_RETRAINING_IN_PROGRESS"

Construct synthetic shifted dataset payload.

Dispatch asynchronous background worker thread:

Train retrained models with increased tree depth & estimators.

```

        Promote newly trained candidate as active Champion Model.
        Reset statistical drift baselines.
        Set state flag = "STABLE_HEALTHY"
    END IF
END

```

□ Prerequisites & System Requirements

- Node.js: v18.0.0 or higher
- Python: v3.10 or higher (Python 3.11 recommended)
- Docker & Docker Compose: (Required for containerized setup) v24.0+
- MongoDB: Local instance running on `mongodb://localhost:27017` or MongoDB Atlas URI

□ Quick-Start Execution Options

Option A: Instant Demo Simulation Mode (Zero Backend Setup)

Ideal for quick evaluation or committee presentation on laptops without Python/MongoDB setup:

1. Install dependencies:

```
``bash cd June/June npm install ``
```

2. Launch the development server:

```
``bash npm run dev ``
```

3. Open `http://localhost:3000` in your browser.

4. Log in using pre-filled credentials. The frontend will run in Demo Simulation Mode, auto-generating telemetry responses, live 5-second polling updates, and full UI interactivity.

Option B: Production Docker Compose Execution

Launches all services (FastAPI Engine, React Web UI, MongoDB, MLflow Registry) in isolated containers:

```
cd June/June
docker-compose up --build -d
```

- React Dashboard: `http://localhost:3000`
- FastAPI REST API Documentation: `http://localhost:8000/docs`
- MLflow Tracking Dashboard: `http://localhost:5001`

Option C: Native Development Environment Setup

1. Start MongoDB & MLflow Backend

```
# Start MongoDB locally (port 27017)
mongod --dbpath ./data/db

# Start MLflow Tracking Server (optional, port 5001)
mlflow server --host 0.0.0.0 --port 5001 --backend-store-uri sqlite:///mlflow.db
```

2. Start FastAPI Machine Learning Engine

```
cd June/June/backend

# Create & activate Python virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Start FastAPI server
python3 main.py
```

The FastAPI backend will start on `http://localhost:8000`.

3. Start React Frontend Interface

```
cd June/June

# Install Node modules
npm install

# Start Vite dev server
npm run dev
```

The interface will be live on <http://localhost:3000>.

□ Step-by-Step User Manual

Step 1: User Registration & Authentication

1. Navigate to <http://localhost:3000>.
2. Click Register if creating a new account, or enter default credentials (admin@pipelineiq.io / admin123).
3. Upon submitting, FastAPI validates credentials against MongoDB (or demo storage) and issues an encrypted JWT Bearer Token valid for 24 hours.

Step 2: Tabular Dataset Upload & AutoML Training

1. On the Dashboard Overview, click the + New Pipeline Setup button in the header.
2. Drag and drop any tabular CSV dataset (e.g. ibm_hr_attrition.csv) into the upload dropzone, or leave it blank to execute using the built-in synthetic telemetry dataset.
3. Select the machine learning models to include in concurrent execution:
 - [x] Random Forest Engine
 - [x] XGBoost Engine
 - [x] LightGBM Engine
4. Choose your primary evaluation target metric (F1-Score, Accuracy, ROC-AUC, Precision, or Recall).
5. Click Launch Training Pipeline. The system dispatches an asynchronous worker task and injects a temporary yellow Training... badge into the live telemetry runs list.

Step 3: Real-Time Telemetry & Champion Model Selection

1. The Telemetry Overview updates live every 5 seconds without manual page reloads using TanStack Query polling.
2. Once background training completes, candidate metrics are displayed. The model with the highest target metric score is automatically crowned with a green CHAMPION MODEL badge.
3. Detailed hyperparameters (n_estimators, max_depth, smote_applied: True, execution runtime) can be inspected by expanding the run card.

Step 4: Statistical Drift Detection & Telemetry Monitoring

1. Scroll down to the Evidently AI Drift Monitoring panel.
2. Under normal baseline conditions, the Statistical Health indicator displays a green STABLE state with Population Stability Index $\text{PSI} < 0.10$ and $p\text{-value} > 0.05$.

Step 5: Injecting Synthetic Covariate Shift & Self-Healing Trigger

1. To demonstrate autonomous self-healing, click the Inject Covariate Shift button in the Drift Monitor card.
2. The drift engine modifies underlying feature distributions, causing the calculated PSI score to spike above the critical boundary of 0.25 ($\text{PSI} \approx 0.38$).
3. The dashboard turns into a high-visibility warning state with a yellow border.
4. An automated alert banner appears: CRITICAL DATA DRIFT DETECTED — AUTONOMOUS RETRAINING IN PROGRESS.

Step 6: Autonomous Retraining & Self-Healing Recovery

1. PipelineIQ autonomously dispatches Algorithm 4 on a background thread.
2. The background engine tunes model hyperparameter depths, retrains candidate classifiers on the shifted data distribution, and promotes the newly optimized champion model.
3. Once retraining completes, click Reset Drift Baseline. The platform restores the status indicator to STABLE / HEALTHY without operational downtime.

REST API Specification

FastAPI automatically serves interactive Swagger UI documentation at <http://localhost:8000/docs>.

Key Endpoints:

HTTP Method	Path	Auth Required	Description
POST	/api/v1/auth/register	No	Registers new operator account & initializes MLflow user profile.
POST	/api/v1/auth/login	No	Authenticates credentials and returns JWT Bearer access token.
GET	/api/v1/health	No	System health check (API, Database, MLflow, Evidently engine status).
POST	/api/v1/models/train	Bearer Token	Dispatches asynchronous AutoML dataset ingestion & training pipeline.
GET	/api/v1/runs	Bearer Token	Fetches live MLflow telemetry runs history for active operator.
GET	/api/v1/drift	Bearer Token	Calculates & returns Kolmogorov-Smirnov and PSI data drift telemetry.
POST	/api/v1/drift/inject	Bearer Token	Injects artificial statistical covariate drift into pipeline runs.
POST	/api/v1/drift/reset	Bearer Token	Resets statistical drift metrics to baseline status.

Database Schema Reference

MongoDB Collections (pipelineiq database):

1. users Collection

```
{
  "_id": "ObjectId('65d21a9f8e4b1029c83a1001')",
  "name": "Operator Admin",
  "email": "admin@pipelineiq.io",
  "password": "$2b$12$elmiTXuWVxfM37uY4JANjO5E/6j5Z8yF7J9...",
  "role": "User",
  "created_at": "2026-08-11T12:00:00Z"
}
```

2. runs Collection

```
{
  "_id": "ObjectId('65d21b108e4b1029c83a1002')",
  "user_id": "65d21a9f8e4b1029c83a1001",
  "id": "run_ibm_active",
  "dataset": "ibm_hr_attrition.csv",
  "models": ["Random Forest", "XGBoost", "LightGBM"],
  "champion_model": "XGBoost Engine",
}
```

```

"hyperparameters": {
  "n_estimators": 200,
  "max_depth": 10,
  "smote_applied": true
},
"status": "Healthy",
"targetMetric": "F1-Score",
"score": 0.942,
"accuracy": 0.951,
"f1_score": 0.942,
"roc_auc": 0.968,
"timestamp": "2026-08-11 12:05:32",
"created_at": "2026-08-11T12:05:32Z"
}

```

□ Troubleshooting & FAQs

Q1: The backend fails with `AttributeError: module 'bcrypt' has no attribute '__about__'`

Solution: Re-install passlib and bcrypt in your Python virtual environment:

```

pip uninstall passlib bcrypt -y
pip install passlib bcrypt==4.0.1

```

Q2: MongoDB connection refusers (`pymongo.errors.ServerSelectionTimeoutError`)

Solution: Ensure your MongoDB daemon is running locally on port 27017 or set the `MONGO_URL` environment variable in `backend/.env`:

```

MONGO_URL=mongodb://localhost:27017

```

Q3: XGBoost fails to install or compile on macOS

Solution: Ensure OpenMP compiler libraries are installed via Homebrew:

```

brew install libomp

```

□ PDF User Manual

A complete, professionally typeset PDF User Manual containing formal mathematical equations, system architecture diagrams, and printable operator guides is available in this repository:

□ [Download PipelineIQ User Manual (PDF)](file:///Users/dev-momin/June/June/PipelineIQ_User_Manual.pdf)

□ Academic Thesis Citation

```

@thesis{PipelineIQ2026,
  title   = {PipelineIQ: A Self-Healing Cloud-Native MLOps Platform for Automated Model Monitoring and Drift Remediation},
  author  = {Manzoor, Maimoona and Andrabi, Syed Maryam and Zahoor, Momin},
  supervisor = {Sholla, Dr. Sahil},
  school  = {Islamic University of Science and Technology (IUST), Awantipora},
  department = {Department of Computer Science and Engineering},
  year    = {2026},
  type    = {B.Tech Thesis Project}
}

```